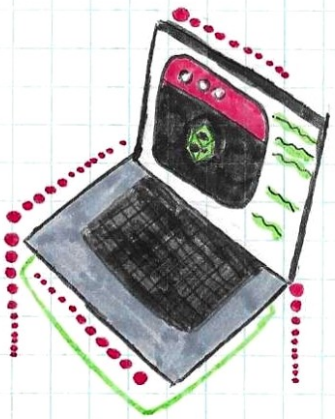
On a aussi la pollution de parametre HTTP ou (HPP) qui consiste à diviser la charge utile en plusieurs valeurs d'un même parametre dans l'URL. Selon l'architecture la plateforme cible peux concacteiner ces fragement et ainsi xscreen et exécuter le payload: (par exemple: ?id=1&id=UNION&id=SELECT&id=username ...)

Exploitation de l'OS:

SQLMap possède la capacité (si notre niveau de privilège le permet) de lire et ecrire des fichiers depuis le system local en dehors du SGBD.
A Noter que les privilèges de lectures sont bien plus courant que d'ecriture...
Nous avons déja vu comment verifier nos privilèges actuels..
Pour ecrire un fichier sur le system distant cela se fais en combinant deux options. Example: --file-write "shell.php" --file-dest "/var/www/html/shell.php"
Pour obtenir un shell OS avec SQLMap sans ecriture complexes si toute les conditions sont reunies on à les option --os-shell_ (--file-read "/etc/passwd" Ex de lecture
--flush-session: efface ancienne session SQLMap automatiquement recréer...



Cross-Site Scripting (XSS)

Catégorie: ⚔ Offensive
Difficulté: Easy, Cur II
Durée Estimé: 6 Heures

Introductions aux XSS:

L'XSS est une vulnérabilité web courante ou un attaquant injecte du code Javascript malveillant dans une page web. Cela se produit lorsque l'application ne nettoi pas correctement les entrés utilisateurs. Le script s'éxécute ensuite dans le navigateur de l'utilisateur qui consulte la page.
Ces attaques peuvent être utilisées pour voler des informations sensibles, forcer des changements de mot de pass et même dans certains cas miner des cryptos...

L'XSS_Stockée:

Le premier type d'XSS et le plus critique est l'XSS stockée. Notre charge utile injecté est stockée dans la base de donnée backend, et répercutée lors de la visite de la page, notre attaque est persistante et peux toucher tout les visiteurs de la pages, ce qui la rend plus critique c'est ca, sa surface d'impact.
Beaucoup de payload peuvent être bloqué par le navigateur ou un WAF malgré la présence de vuln rappelons une super doc à payloads: PayloadsAllTheThings

- Pour voir si la charge util est persistante et stockée sur le Backend on peu actualiser la page et voir si le XSS injecté est toujours exécuté... Si oui on est bien en présence d'une XSS stockée/Persistante

Example de payload basic: <script>alert(document.cookie);</script>

L'XSS_Reflétée:

Cette vulnerabilité se produit lorsque notre entrée atteint le serveur backend et nous est retournée sans être filtrer ou assainis, si on rafraichi la page les pay